

HW05_Report

Comparison of Tree Heights

1. **Ascending order**
2. **Random order**
3. **Descending order**

Observations

Tree Type	Ascending Order	Random Order	Descending Order
SimpleBST	Very tall (n)	Moderate	Very tall (n)
AVL	Logarithmic	Logarithmic	Logarithmic
Red-Black	Slightly taller than AVL	Close to log(n)	Slightly taller than AVL

- **SimpleBST:** Unbalanced insertion orders (ascending/descending) create highly skewed trees.
- **AVL:** Self-balancing ensures tree height remains approximately $\log_2(n)$ regardless of insertion order.
- **Red-Black:** Maintains balance using color rules; height can be slightly greater than AVL, but remains $O(\log n)$.

Time Complexity Observations in Practice

- **Insertion:**
 - SimpleBST: $O(n)$ in worst case (skewed tree), $O(\log n)$ on average for random input.
 - AVL: $O(\log n)$ consistently due to rotations on imbalance.

- Red-Black: $O(\log n)$ consistently; fewer rotations on average compared to AVL.
- **Deletion:**
 - SimpleBST: $O(n)$ worst-case.
 - AVL: $O(\log n)$ with occasional multiple rotations.
 - Red-Black: $O(\log n)$ with color-fixup operations; generally fewer rotations than AVL.
- **Search/Contains:**
 - All trees have $O(\log n)$ in balanced trees; SimpleBST can degrade to $O(n)$ for skewed inputs.

Overall, AVL trees are slightly stricter in balance, which can result in more rotations during insertions and deletions, whereas Red-Black trees allow slightly more relaxed balance, improving performance in practice for large datasets.

Handling Edge Cases

Insertion

- **Duplicates:** Our implementations ignore duplicate values. Only unique elements are inserted.
- **Empty Tree:** Inserting into an empty tree correctly sets the root.

Deletion

- **Deleting the Root:** Handled correctly by transplanting the replacement node (successor for AVL, successor or predecessor for RB).
- **Leaf Node:** Deleted directly.
- **Node with One Child:** Child replaces the node.
- **Node with Two Children:**
 - AVL uses in-order successor.

- Red-Black can use either successor or predecessor (configurable).
- Ensures all child pointers and parent references are updated properly, preserving tree invariants.

Red-Black Specifics

- Uses sentinel NIL nodes for null children to simplify balancing and fix-up operations.
- Root is always black; no red node has a red child; black-height is consistent across paths.

Summary

- **SimpleBST**: Simple but height-sensitive to insertion order.
- **AVL**: Strictly balanced, consistent $O(\log n)$ operations, but more rotations.
- **Red-Black**: Slightly relaxed balancing, fewer rotations, consistently good performance.

These observations confirm theoretical time complexities and demonstrate how self-balancing trees improve practical performance on unbalanced input sequences.